

LAPORAN PRAKTIKUM PEKAN 3

PEMOGRAMAN BERORIENTASI OBJEK



Disusun oleh:

ANNAFI AL GHIFARI

2511533013

Dosen pengampu

Nurfiah, S.ST, M.Kom.

Asisten Praktikum

Dawi Syauqi

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG

2026

DAFTAR ISI

PROGRAM.....	3
KESIMPULAN.....	12

PROGRAM

2.2 Program Main.java

2.2.1 Kode Program Main.java

```
package pekan3;

import java.util.Scanner;

public class Main {

    static Scanner input = new
Scanner(System.in);
    static Rekening akunAktif = null;

    public static void main(String[] args) {
        int pilihan;

        do {
            tampilkanMenu();
            System.out.print("Pilih menu: ");
            pilihan = bacaAngka();

            switch (pilihan) {
                case 1:
                    bukaRekening();
                    break;
                case 2:
                    setorTunai();
                    break;
                case 3:
                    tarikTunai();
                    break;
                case 4:
                    cekSaldo();
                    break;
                case 5:
                    cetakInfoRekening();
                    break;
            }
        } while (pilihan != 0);
    }
}
```

```

        case 6:
            cetakMutasi();
            break;
        case 7:
            System.out.println("Terima kasih telah menggunakan layanan kami. Sampai
jumpa!");
            break;
        default:
            System.out.println("Pilihan tidak valid. Silakan coba lagi.");
    }

    System.out.println();

} while (pilihan != 7);

input.close();
}

static void tampilkanMenu() {
    System.out.println("==== MENU BANK =====");
    System.out.println("1. Buka Rekening");
    System.out.println("2. Setor Tunai");
    System.out.println("3. Tarik Tunai");
    System.out.println("4. Cek Saldo");
    System.out.println("5. Info Rekening");
    System.out.println("6. Cetak Mutasi (Riwayat Transaksi)");
    System.out.println("7. Keluar");
}

static int bacaAngka() {
    while (!input.hasNextInt()) {
        System.out.print("Masukkan angka yang valid: ");
        input.next();
    }
    int angka = input.nextInt();
    input.nextLine(); // buang sisa newline
    return angka;
}

```

```

// 1. Buka Rekening -- minta PIN sebelum instansiasi objek baru
static void bukaRekening() {
    System.out.print("Masukkan nomor rekening: ");
    String nomor = input.nextLine();

    System.out.print("Masukkan nama pemilik: ");
    String nama = input.nextLine();

    System.out.print("Masukkan saldo awal: ");
    double saldoAwal = Double.parseDouble(bacaAngkaDesimal());

    String pin;
    while (true) {
        System.out.print("Masukkan PIN (6 digit angka): ");
        pin = input.nextLine();
        if (pin.length() == 6 && pin.chars().allMatch(Character::isDigit)) {
            break;
        }
        System.out.println("PIN tidak valid! PIN harus terdiri dari 6 digit angka.");
    }

    akunAktif = new Rekening(nomor, nama, saldoAwal, pin);
}

static String bacaAngkaDesimal() {
    while (!input.hasNextDouble()) {
        System.out.print("Masukkan angka yang valid: ");
        input.next();
    }
    String nilai = String.valueOf(input.nextDouble());
    input.nextLine(); // buang sisa newline
    return nilai;
}

// 2. Setor Tunai
static void setorTunai() {
    if (!adaAkunAktif()) return;

    System.out.print("Masukkan nominal setor: ");
    double nominal = Double.parseDouble(bacaAngkaDesimal());
    akunAktif.setor(nominal);
}

// 3. Tarik Tunai -- wajib otentikasi PIN dulu
static void tarikTunai() {
    if (!adaAkunAktif()) return;

    System.out.print("Masukkan PIN Anda: ");
    String pinYangDiinput = input.nextLine();

    if (akunAktif.otentikasi(pinYangDiinput)) {
        System.out.print("Masukkan nominal tarik: ");
        double nominal = Double.parseDouble(bacaAngkaDesimal());
        akunAktif.tarik(nominal);
    } else {
        System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
    }
}

// 4. Cek Saldo
static void cekSaldo() {
    if (!adaAkunAktif()) return;
    System.out.println("Saldo saat ini: Rp" + akunAktif.getSaldo());
}

// 5. Info Rekening
static void cetakInfoRekening() {
    if (!adaAkunAktif()) return;
    akunAktif.cetakInfo();
}

// 6. Cetak Mutasi -- wajib otentikasi PIN dulu
static void cetakMutasi() {
    if (!adaAkunAktif()) return;

    System.out.print("Masukkan PIN Anda: ");
    String pinYangDiinput = input.nextLine();

    if (akunAktif.otentikasi(pinYangDiinput)) {
        System.out.println("---- Riwayat Transaksi ----");
        if (akunAktif.getRiwayatTransaksi().isEmpty()) {
            System.out.println("Belum ada transaksi.");
        } else {
            for (Transaksi t : akunAktif.getRiwayatTransaksi()) {
                t.cetakDetail();
            }
        }
    } else {
        System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
    }
}

static boolean adaAkunAktif() {
    if (akunAktif == null) {
        System.out.println("Belum ada rekening aktif. Silakan buka rekening terlebih dahulu (Menu 1).");
        return false;
    }
    return true;
}
}

```

2.2.2 Penjelasan

1. Class Transaksi berada pada package pekan3 dan berperan sebagai class sederhana untuk merepresentasikan satu objek transaksi (data holder), dengan tiga atribut: idTransaksi, jenis, dan nominal. Ketiga atribut ini menggunakan modifier akses default (package-private), yaitu hanya dapat diakses dari class lain yang berada pada package yang sama.
2. Constructor Transaksi(String id, String jenis, double nominal) menginisialisasi ketiga atribut menggunakan kata kunci this untuk membedakan atribut class dengan parameter yang memiliki nama sama.
3. Disediakan tiga getter method publik (getIdTransaksi(), getJenis(), getNominal()) yang memungkinkan class lain (seperti Main) membaca nilai atribut objek Transaksi tanpa mengubahnya secara langsung.
4. Method cetakDetail() mencetak representasi satu baris dari objek transaksi, yang dipanggil dari Main.cetakMutasi() untuk menampilkan setiap item riwayat transaksi.
5. Class ini tidak memiliki atribut private maupun banyak validasi karena perannya murni sebagai objek data yang dibuat dan disimpan oleh class Rekening setiap kali terjadi transaksi setor atau tarik.

2.3 Program Rekening.java

2.3.1 Kode Program Rekening.java

```
package pekan3;

import java.util.ArrayList;

public class Rekening {
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin;

    private ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        if (pinAwal.length() == 6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456");
            this.pin = "123456";
        }
        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
    }

    public String getNomorRekening() {
        return nomorRekening;
    }
}
```

```
    public String getNamaPemilik() {
        return namaPemilik;
    }

    public double getSaldo() {
        return saldo;
    }

    public ArrayList<Transaksi> getRiwayatTransaksi() {
        return riwayatTransaksi;
    }

    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }

    public boolean setor(double nominal) {
        if (nominal <= 0) {
            System.out.println("Nominal setor harus lebih dari 0.");
            return false;
        }
        this.saldo += nominal;
        String id = "TRX" + (riwayatTransaksi.size() + 1);
        riwayatTransaksi.add(new Transaksi(id, "Setor", nominal));
        System.out.println("Setor Rp" + nominal + " berhasil. Saldo sekarang: Rp" + saldo);
        return true;
    }
}
```

```

public boolean tarik(double nominal) {
    if (nominal <= 0) {
        System.out.println("Nominal tarik harus lebih dari 0.");
        return false;
    }
    if (nominal > saldo) {
        System.out.println("Saldo tidak cukup.");
        return false;
    }
    this.saldo -= nominal;
    String id = "TRX" + (riwayatTransaksi.size() + 1);
    riwayatTransaksi.add(new Transaksi(id, "Tarik", nominal));
    System.out.println("Tarik Rp" + nominal + " berhasil. Saldo sekarang: Rp" + saldo);
    return true;
}

public void cetakInfo() {
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Pemilik      : " + namaPemilik);
    System.out.println("Saldo        : Rp" + saldo);
}
}

```

2.3.2 Penjelasan

1. Class Rekening mendeklarasikan seluruh atributnya (nomorRekening, namaPemilik, saldo, pin, riwayatTransaksi) sebagai private, sehingga menerapkan prinsip enkapsulasi secara penuh — atribut ini hanya dapat diakses dari luar class melalui getter method atau method publik lain yang telah disediakan.
2. Atribut riwayatTransaksi bertipe ArrayList<Transaksi> menunjukkan relasi komposisi: satu objek Rekening memiliki dan mengelola banyak objek Transaksi sekaligus.
3. Constructor Rekening(...) menginisialisasi nomor rekening, nama pemilik, dan saldo awal, kemudian memvalidasi PIN yang dimasukkan: jika panjangnya tepat 6 karakter maka digunakan sebagai PIN, jika tidak maka program mencetak peringatan dan menetapkan PIN default "123456". Di akhir constructor, riwayatTransaksi diinisialisasi sebagai ArrayList kosong, dan pesan konfirmasi pembuatan rekening dicetak.
4. Method otentikasi(String inputPin) membandingkan PIN tersimpan dengan PIN yang diinput menggunakan equals(), dan mengembalikan nilai boolean

sebagai dasar validasi sebelum operasi sensitif (tarik tunai dan cetak mutasi) dapat dilakukan.

5. Method `setor(double nominal)` dan `tarik(double nominal)` masing-masing memvalidasi nominal transaksi (harus > 0 , dan untuk tarik tunai saldo harus mencukupi) sebelum memperbarui saldo, membuat objek Transaksi baru dengan ID otomatis ("TRX" + urutan), menambahkannya ke riwayatTransaksi, lalu mencetak konfirmasi. Kedua method mengembalikan nilai boolean untuk menandakan keberhasilan transaksi.

6. Method `cetakInfo()` menampilkan ringkasan data rekening (nomor, pemilik, saldo) yang dipanggil dari `Main.cetakInfoRekening()`.

2.4 Program Transaksi.java

2.4.1 Kode Program Transaksi.java

```
package pekan3;

public class Transaksi {
    String idTransaksi;
    String jenis;
    double nominal;

    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    public String getIdTransaksi() { return idTransaksi; }
    public String getJenis() { return jenis; }
    public double getNominal() { return nominal; }

    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" +
            nominal);
    }
}
```

2.4.2 Penjelasan

1. Atribut static: input (Scanner) dan akunAktif (objek Rekening yang sedang login, null di awal).

2. `main()`: loop do-while yang memanggil `tampilkanMenu()` → `bacaAngka()` untuk baca pilihan → switch-case menjalankan method sesuai nomor menu, berulang sampai pilihan = 7 (Keluar).
3. `bacaAngka()/bacaAngkaDesimal()`: validasi input agar hanya menerima angka, memakai loop while (`!input.hasNextInt()`) sehingga input salah tidak membuat program *crash*.
4. `bukaRekening()` (Menu 1): mengumpulkan data, validasi PIN dalam `while(true)` sampai valid (6 digit angka via `pin.chars().allMatch(Character::isDigit)`), baru kemudian `new Rekening(...)` dipanggil — di sinilah constructor `Rekening` dieksekusi.
5. `setorTunai()` / `tarikTunai()` / `cekSaldo()` / `cetakInfoRekening()` / `cetakMutasi()` (Menu 2–6): masing-masing cek dulu `adaAkunAktif()` (mencegah `NullPointerException` bila rekening belum dibuat), lalu memanggil method terkait pada `akunAktif`. Khusus `tarikTunai()` dan `cetakMutasi()`, wajib lolos `akunAktif.otentikasi(pin)` dulu. Di `cetakMutasi()`, program melakukan for (Transaksi t : `akunAktif.getRiwayatTransaksi()`) untuk mencetak setiap riwayat transaksi satu per satu.
6. `adaAkunAktif()`: helper yang mengecek `akunAktif == null`, mencegah error saat pengguna memilih menu 2–6 sebelum membuka rekening.

2.5 Output

Ketika membuka rekening dia meminta PIN 6 Digit

```
===== MENU BANK =====
1. Buka Rekening
2. Setor Tunai
3. Tarik Tunai
4. Cek Saldo
5. Info Rekening
6. Cetak Mutasi (Riwayat Transaksi)
7. Keluar
Pilih menu: 1
Masukkan nomor rekening: 123
Masukkan nama pemilik: nafi
Masukkan saldo awal: 10000
Masukkan PIN (6 digit angka): 111111
Rekening atas nama nafi berhasil dibuat.
```

Ketika Tarik tunai, dia akan meminta PIN sebelum memasukkan nominal tarik

```
===== MENU BANK =====
1. Buka Rekening
2. Setor Tunai
3. Tarik Tunai
4. Cek Saldo
5. Info Rekening
6. Cetak Mutasi (Riwayat Transaksi)
7. Keluar
Pilih menu: 3
Masukkan PIN Anda: 111111
Masukkan nominal tarik: 5000
Tarik Rp5000.0 berhasil. Saldo sekarang: Rp5000.0
```

Ketika mencetak mutase (Riwayat Transaksi) dia akan meminta pin terlebih dahulu

```
===== MENU BANK =====
1. Buka Rekening
2. Setor Tunai
3. Tarik Tunai
4. Cek Saldo
5. Info Rekening
6. Cetak Mutasi (Riwayat Transaksi)
7. Keluar
Pilih menu: 6
Masukkan PIN Anda: 111111
--- Riwayat Transaksi ---
ID: TRX1 | Jenis: Tarik | Nominal: Rp5000.0
```

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan pada studi kasus sistem perbankan sederhana, dapat disimpulkan beberapa hal sebagai berikut:

1. Konsep class dan object berhasil diterapkan dengan baik melalui class Rekening dan Transaksi, di mana objek-objeknya dibuat (diinstansiasi) secara dinamis pada saat program berjalan sesuai interaksi pengguna.
2. Prinsip enkapsulasi diterapkan secara konsisten pada class Rekening dengan mendeklarasikan seluruh atributnya sebagai private dan menyediakan getter method publik, sehingga data rekening (khususnya saldo dan PIN) terlindungi dari akses maupun perubahan langsung dari luar class.
3. Constructor pada class Rekening tidak hanya menginisialisasi nilai atribut, tetapi juga melakukan validasi data (panjang PIN), membuktikan bahwa constructor dapat digunakan untuk menjaga konsistensi data sejak objek pertama kali dibuat.
4. Relasi komposisi antara Rekening dan Transaksi yang diimplementasikan menggunakan ArrayList<Transaksi> terbukti berhasil menyimpan dan menampilkan riwayat transaksi secara berurutan, menunjukkan bagaimana satu objek dapat mengelola banyak objek lain sebagai bagian dari dirinya.
5. Secara keseluruhan, program simulasi sistem perbankan ini berhasil mengintegrasikan tiga class yang saling berelasi untuk membentuk sebuah aplikasi konsol yang fungsional, sekaligus menjadi sarana pemahaman praktis terhadap konsep dasar Pemrograman Berorientasi Objek dalam bahasa Java.